Lab 11 –
Dictionaries in Python

Programming Exercise:

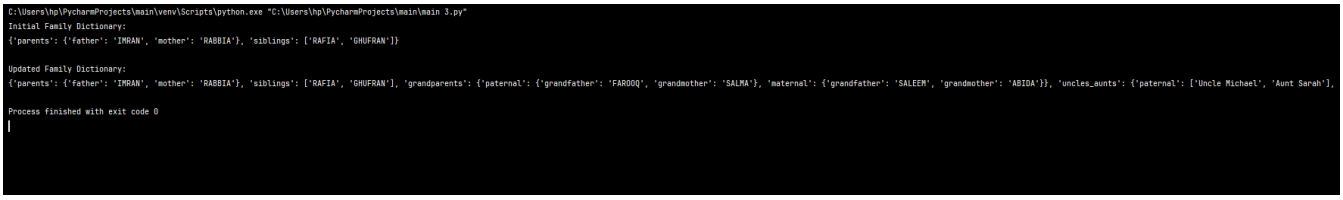
1. Design a dictionary of your family. Once you get the printout update family dictionary with your grandparents (maternal and paternal) including uncles and aunts (maternal and paternal).

Source Code:

```
# Initial family dictionary
family = {
    "parents": {"father": "IMRAN",
                "mother": "RABBIA"},
    "siblings": ["RAFIA", "GHUFRAN"]
        }

# Print the initial family dictionary
print("Initial Family Dictionary:")
print(family)

# Update the family dictionary to include grandparents and uncles/aunts
family.update({
    "grandparents": {
        "paternal": {
            "grandfather": "FAROOQ",
            "grandmother": "SALMA"
        },
        "maternal": {
            "grandfather": "SALEEM",
            "grandmother": "ABIDA"
        }
```

```
    },
    "uncles_aunts": {
        "paternal": ["Uncle Michael", "Aunt Sarah"],
        "maternal": ["Uncle James", "Aunt Linda"]
    }
})

# Print the updated family dictionary
print("\nUpdated Family Dictionary:")
print(family)
```

Output:



2. Write a function to design a personal phone directory of your parents and friends. You must add 12 members. Then make a function to delete a member from a telephone directory. Print total number of members in your personal phone directory.

Source Code:

```python
# Function to create the initial phone directory with 12 members
def create_phone_directory():  1 usage
    phone_directory = {
        "John (Father)": "123-456-7890",
        "Jane (Mother)": "234-567-8901",
        "Alice (Friend)": "345-678-9012",
        "Bob (Friend)": "456-789-0123",
        "Charlie (Friend)": "567-890-1234",
        "Dave (Friend)": "678-901-2345",
        "Eve (Friend)": "789-012-3456",
        "Frank (Friend)": "890-123-4567",
        "Grace (Friend)": "901-234-5678",
        "Hannah (Friend)": "012-345-6789",
        "Ivy (Friend)": "123-456-7800",
        "Jack (Friend)": "234-567-8900"
    }
    return phone_directory

# Function to delete a member from the phone directory
def delete_member(phone_directory, name):  1 usage
    if name in phone_directory:
        del phone_directory[name]
        print(f"{name} has been removed from the phone directory.")
    else:
        print(f"{name} not found in the phone directory.")

# Function to print the total number of members in the directory
def total_members(phone_directory):  1 usage
    return len(phone_directory)

# Main code
phone_directory = create_phone_directory()
print("Initial Phone Directory:", phone_directory)

# Delete a member
delete_member(phone_directory, name: "Alice (Friend)")

# Print updated directory and total count of members
print("\nUpdated Phone Directory:", phone_directory)
print("Total number of members:", total_members(phone_directory))
```

Output:

```
C:\Users\hp\PycharmProjects\main\venv\Scripts\python.exe "C:\Users\hp\PycharmProjects\main\main 3.py"
Initial Phone Directory: {'John (Father)': '123-456-7890', 'Jane (Mother)': '234-567-8901', 'Alice (Friend)': '345-678-9012'
Alice (Friend) has been removed from the phone directory.

Updated Phone Directory: {'John (Father)': '123-456-7890', 'Jane (Mother)': '234-567-8901', 'Bob (Friend)': '456-789-0123',
Total number of members: 11

Process finished with exit code 0
```

```
!', 'Bob (Friend)': '456-789-0123', 'Charlie (Friend)': '567-890-1234', 'Dave (Friend)': '678-901-2345', 'Eve (Friend)': '789-012-3456',


 'Charlie (Friend)': '567-890-1234', 'Dave (Friend)': '678-901-2345', 'Eve (Friend)': '789-012-3456', 'Frank (Friend)': '890-123-4567',
```

```
'Eve (Friend)': '789-012-3456', 'Frank (Friend)': '890-123-4567', 'Grace (Friend)': '901-234-5678', 'Hannah (Friend)': '012-345-6789', 'Ivy (Friend)': '123-456-7800', 'Jack (Friend)': '234-567-8900'}


rank (Friend)': '890-123-4567', 'Grace (Friend)': '901-234-5678', 'Hannah (Friend)': '012-345-6789', 'Ivy (Friend)': '123-456-7800', 'Jack (Friend)': '234-567-8900'}
```

3. Write a function hexASCII() that prints the correspondence between the lowercase characters in the alphabet and the hexadecimal representation of their ASCII code. Note: [A format string and the format string method can be used to represent a number value in hex notation.]

Source Code:

```
def hexASCII():  1 usage
    for char in range(ord('a'), ord('z') + 1):
        hex_value = format(char, 'x')  # Convert ASCII value to hexadecimal
        print(f"{chr(char)} : {hex_value}")

# Call the function
hexASCII()
```

Output:

```
C:\Users\hp\PycharmProjects\main\venv\Sc
a : 61
b : 62
c : 63
d : 64
e : 65
f : 66
g : 67
h : 68
i : 69
j : 6a
k : 6b
l : 6c
m : 6d
n : 6e
o : 6f
p : 70
q : 71
r : 72
s : 73
t : 74
u : 75
v : 76
w : 77
x : 78
y : 79
z : 7a

Process finished with exit code 0
```

4. Create double dictionaries one of which is your choice of dishes. Other one is dishes cooked in a week. Compare them and find how many dishes you will get of your choice to be cooked in next week. Print the name of those dishes as well

Source Code:

```python
# Dictionary for your choice of dishes
choice_of_dishes = {
    "Monday": "Pasta",
    "Tuesday": "Pizza",
    "Wednesday": "Biryani",
    "Thursday": "Tacos",
    "Friday": "Sushi",
    "Saturday": "Burger",
    "Sunday": "Salad"}

# Dictionary for dishes cooked this week
dishes_cooked_in_week = {
    "Monday": "Pasta",
    "Tuesday": "Sandwich",
    "Wednesday": "Biryani",
    "Thursday": "Noodles",
    "Friday": "Sushi",
    "Saturday": "Pizza",
    "Sunday": "Soup"}

# Find common dishes that match the choice of dishes
matching_dishes = {day: dish for day, dish in choice_of_dishes.items() if dishes_cooked_in_week.get(day) == dish}

# Print the number and names of dishes that match
print("Number of your choice dishes to be cooked next week:", len(matching_dishes))
print("Dishes to be cooked next week from your choices:")

for day, dish in matching_dishes.items():
    print(f"{day}: {dish}")
```

Output:

```
C:\Users\hp\PycharmProjects\main\venv\Scripts\python.exe "C:\Users\hp\PycharmProjects\main\main 3.py"
Number of your choice dishes to be cooked next week: 3
Dishes to be cooked next week from your choices:
Monday: Pasta
Wednesday: Biryani
Friday: Sushi

Process finished with exit code 0
```

5. Design a list of guests with family members on your sister wedding. Each family members must be counted. Your parents have made a list of guests and you have made another list. At the end compare both the list and find the common guests which both of you have invited and count them once. The program will return the number of guest with members and total number of guest. Use functions to perform the required actions.

Source Code:

```python
# Function to count total members from a guest list
def count_members(guest_list):  1 usage
    total_members = 0
    for family, members in guest_list.items():
        total_members += members
    return total_members

# Function to find common guests
def find_common_guests(your_list, parents_list):  1 usage
    common_guests = {guest: min(your_list[guest], parents_list[guest])
                     for guest in your_list if guest in parents_list}
    return common_guests

# Function to combine guest lists, counting each family once
def combine_guest_lists(your_list, parents_list):  1 usage
    combined_list = parents_list.copy()
    combined_list.update({guest: max(your_list.get(guest, 0), parents_list.get(guest, 0))
                          for guest in your_list})
    return combined_list

# Main program
# Sample guest list created by you and your parents with family members count
your_guest_list = {
    "SM Family": 5,
    "RM Family": 3,
    "FM Family": 4,
    "BB Family": 2,
    "JR Family": 6
}

parents_guest_list = {
    "SM Family": 5,
    "RM Family": 3,
    "FM Family": 4,
    "CL Family": 2,
    "JR Family": 6
}
```

```
38
39  # Find common guests and combined list
40  common_guests = find_common_guests(your_guest_list, parents_guest_list)
41  combined_guest_list = combine_guest_lists(your_guest_list, parents_guest_list)
42
43  # Count total members in combined guest list
44  total_members_combined = count_members(combined_guest_list)
45
46  # Print results
47  print("Common guests (invited by both):", common_guests)
48  print("Total number of common guests:", len(common_guests))
49  print("Total number of unique guests:", len(combined_guest_list))
50  print("Total number of members attending:", total_members_combined)
```

Output:

```
C:\Users\hp\PycharmProjects\main\venv\Scripts\python.exe "C:\Users\hp\PycharmProjects\main\main 3.py"
Common guests (invited by both): {'SM Family': 5, 'RM Family': 3, 'FM Family': 4, 'JR Family': 6}
Total number of common guests: 4
Total number of unique guests: 6
Total number of members attending: 22

Process finished with exit code 0
```